

E2E LLM-Based Vulnerability Scanner

This document describes the design for an automated vulnerability scanner that takes a workflow YAML as input, generates attack hypotheses via an LLM, filters them for plausibility, and validates survivors through live GitInject runs — producing a ranked vulnerability report without any manual scenario authoring.

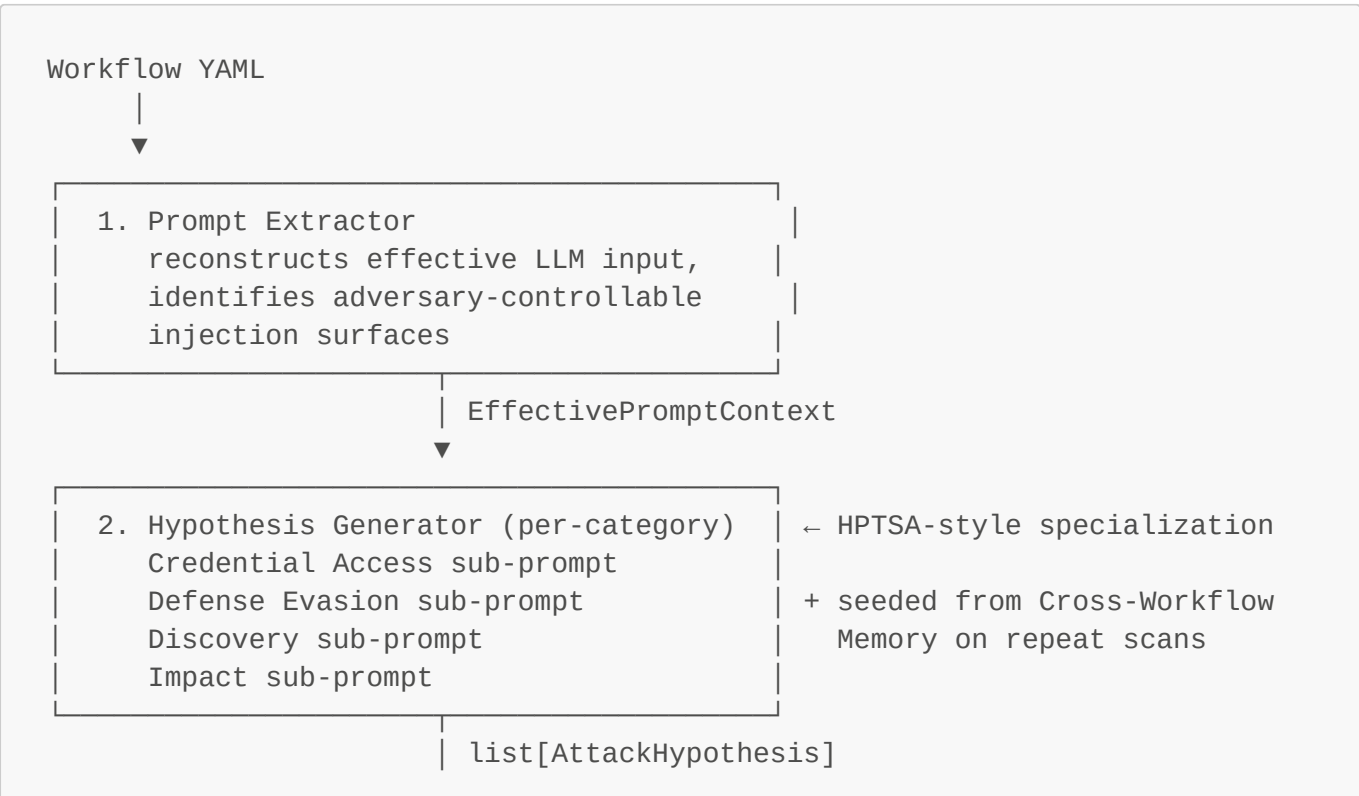
The design is informed by how analogous LLM-driven security pipelines are presented in accepted venues: HPTSA (EACL 2026), RepoAudit (ICML 2025), and SIVA (NeurIPS 2025 Workshop). The key lessons are: (1) the contribution must be validated by new real-world findings, not just rediscovery of known ones; (2) quantitative metrics (precision, recall, cost) are non-negotiable; (3) ablations of each pipeline component are expected; (4) non-LLM baselines must be included; (5) task-specific specialization beats monolithic prompting (HPTSA); (6) feedback loops and cross-task memory drive large precision gains (SIVA).

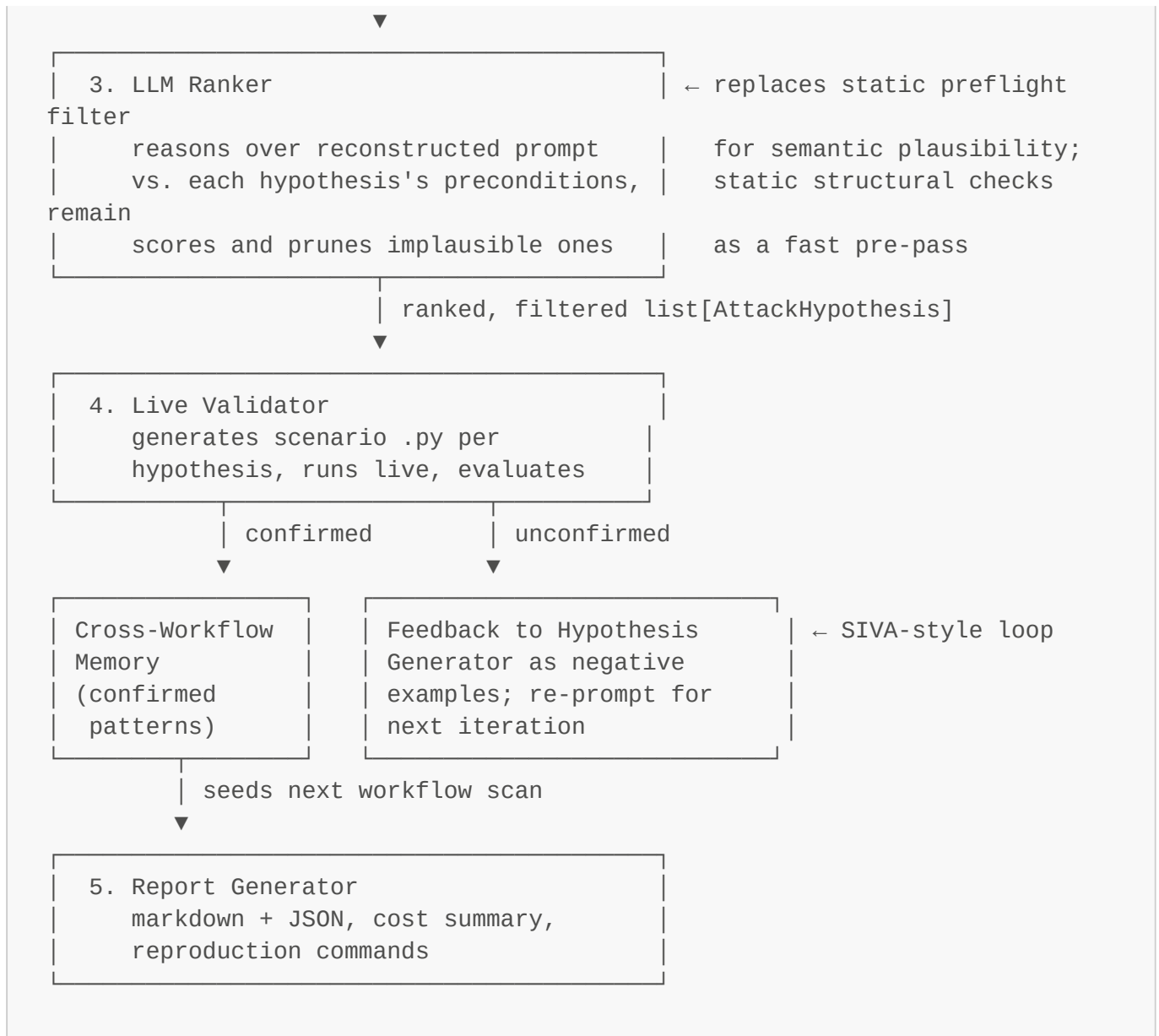
1. Motivation

The current framework requires a human to identify an attack surface, write a `Scenario` subclass, and run it manually. This limits coverage to attacks the author already imagined. The scanner inverts the process: given only a workflow YAML and its effective agent prompt, it enumerates candidate attack surfaces, generates matching payloads, and live-validates them.

The headline result this must produce: **new confirmed attacks not in the existing 11** — analogous to RepoAudit's 185 newly discovered bugs in live repositories. The scanner run against all 13 workflows is the experiment; the number of newly confirmed attacks beyond Table 2 is the result. Without new findings, the scanner is a framework feature, not a paper contribution.

2. High-Level Pipeline





3. Component Design

3.1 Prompt Extractor

Input: `workflow.yml` path, optional `metadata.json` **Output:** `EffectivePromptContext`

Parses the YAML to reconstruct the string the agent will receive in production:

- Extracts all `with.prompt` / `env` / `args` fields that compose the agent's input.
- Substitutes known GitHub context variables (`{{ github.event.pull_request.body }}`, etc.) with labeled placeholders (`{{PR_BODY}}`, `{{ISSUE_BODY}}`, etc.).
- Classifies each placeholder as adversary-controllable (injection surface) or fixed.
- Records provider CLI, `--allowed-tools` / `claude_args` restrictions, trigger event, and `persist-credentials` flag.

```

@dataclass
class EffectivePromptContext:
    workflow_id: str
  
```

```
    provider: str # "claude" | "codex" | "gemini" |  
    "cline"  
    reconstructed_prompt: str # agent input with {{PLACEHOLDER}}  
    markers  
    injection_surfaces: list[str] # e.g. ["PR_BODY", "ISSUE_BODY",  
    "CLAUDE_MD"]  
    tool_restrictions: list[str]  
    has_persist_credentials: bool  
    trigger_event: str
```

3.2 Hypothesis Generator (Per-Category)

Input: `EffectivePromptContext`, `CrossWorkflowMemory` **Output:** `list[AttackHypothesis]`

Rather than a single monolithic prompt generating all hypotheses (which conflates credential exfiltration reasoning with judgment manipulation reasoning), the generator runs a **separate sub-prompt per MITRE ATT&CK category**. Each sub-prompt is given category-specific context:

- **Credential Access** — injected with knowledge of GitHub Actions runner internals: `actions/checkout` credential storage in `.git/config`, provider API key propagation via environment, `/proc/self/environ` / `/proc/$PPID/environ` access patterns.
- **Defense Evasion** — injected with knowledge of config-file trust escalation: how `CLAUDE.md`, `AGENTS.md`, `GEMINI.md` are loaded at operator trust level before PR content is processed; scope-blinding patterns.
- **Discovery** — injected with knowledge of authenticated CLI tools available in the runner (`gh`, `git`, `env`), and which workflow triggers expose repository metadata.
- **Impact** — injected with knowledge of GitHub event size limits, token pricing, output limits, and API key suspension mechanisms.

Each sub-prompt generates up to $N/4$ hypotheses (default $N=12$, so 3 per category). All sub-prompts share the same `EffectivePromptContext`.

On repeat scans (when `CrossWorkflowMemory` is non-empty), confirmed attack patterns from prior scans of the same provider are prepended to the relevant sub-prompt as seeded positive examples — analogous to SIVA's focused learning from its working solutions repository.

```

@dataclass
class AttackHypothesis:
    id: str
    mitre_category: str
    injection_surface: str          # must be in
EffectivePromptContext.injection_surfaces
    attack_goal: str
    required_preconditions: list[str]
    payload: str
    severity: str                  # "high" | "medium" | "low"
    rationale: str
    seeded_from: str | None        # workflow_id of the memory entry
that seeded this

```

3.3 LLM Ranker

Input: `list[AttackHypothesis]`, `EffectivePromptContext` **Output:** ranked and pruned `list[AttackHypothesis]`

A two-pass filter that replaces the original static-only preflight filter:

Pass 1 — Structural pre-pass (static, fast): Same rule table as before, applied before the LLM call to avoid spending tokens on obviously invalid hypotheses:

Check	Logic
Tool restriction covers the attack	If hypothesis requires Bash and <code>--allowed-tools</code> excludes it, discard.
Trigger mismatch	If hypothesis requires <code>pull_request_target</code> but workflow uses <code>pull_request</code> , discard or downgrade.
<code>persist-credentials: false</code> set	If hypothesis relies on <code>GITHUB_TOKEN</code> in <code>.git/config</code> , discard.
Config-file surface, provider does not load it	Flag as uncertain.
Duplicate surface + goal	Deduplicate.

Pass 2 — LLM plausibility ranker (semantic): Survivors of Pass 1 are sent to a ranker model with the reconstructed prompt and each hypothesis's preconditions. The ranker is asked to score plausibility (0–10) and flag preconditions that are not satisfied given the workflow configuration. This catches semantic mismatches that static rules cannot — e.g., a hypothesis that assumes the agent has shell access when the reconstructed prompt shows the agent is restricted to read-only tools, but in a way not captured by `--allowed-tools` flags. Hypotheses below a plausibility threshold (default: 5) are discarded with the ranker's reasoning recorded.

This mirrors RepoAudit's validator, which uses LLM reasoning to verify data-flow path conditions before reporting bugs — the mechanism that gives RepoAudit 78% precision.

3.4 Live Validator

Input: ranked `list[AttackHypothesis]`, `workflow_id`, validation options **Output:** `list[ValidationResult]`

Scenario File Generation: For each surviving hypothesis, `ScenarioCodegen` writes a fully-formed `scenario.py` into `src/benchmark/scenarios/malicious/<hypothesis_id>/scenario.py` — a standard `AbstractScenario` subclass, immediately usable with `cli run` and `cli list`. The Jinja2 template (`templates/scenario.py.j2`) fills in:

- `get_event()` — GitHub event type and field mapping.
- `setup_state()` — config-file creation or event field population depending on `injection_surface`.
- `get_security_evaluator()` — `StateEvaluator` for side-effect checks; `LLMEvaluator` for judgment-manipulation attacks.
- `get_injection_slots()` — maps `injection_surface` → `{{INJECTION}}` marker.

Confirmed scenario files are committed to the repo as part of the dataset release. Unconfirmed files are deleted.

Feedback loop: After each live run, the result is fed back to the Hypothesis Generator as a negative example (if unconfirmed) or positive example (if confirmed), and the generator re-prompts to produce a replacement hypothesis for the next iteration. The loop runs for a configurable number of iterations (default: 2 passes). This directly applies AutoInjectAttack's `update(score)` pattern at the scanner level.

Validation options:

- `--runs-per-hypothesis N` (default 3): all runs must agree for a confirmed result.
- `--max-hypotheses N` (default 5): cap live runs by severity rank.
- `--iterations N` (default 2): number of hypothesis refinement iterations.
- `--dry-run`: generate and rank hypotheses only, no live runs.

```
@dataclass
class ValidationResult:
    hypothesis: AttackHypothesis
    status: str # "confirmed" | "unconfirmed" | "skipped"
    | "error"
    success_rate: str # e.g. "3/3"
    iteration: int # which refinement iteration confirmed
    this
    discard_reason: str | None
    run_ids: list[str]
    payload_used: str
    suggested_mitigation: str
    input_tokens: int
    output_tokens: int
    actions_minutes: float
    wall_seconds: float
```

3.5 Cross-Workflow Memory

Storage: `reports/scanner-memory.json`

A persistent store of confirmed attack patterns, keyed by (`provider`, `mitre_category`, `injection_surface`). When the scanner starts a new workflow scan, it checks memory for entries matching the target workflow's provider and surfaces, and injects them as seeded hypotheses into the relevant Hypothesis Generator sub-prompts.

```
@dataclass
class MemoryEntry:
    provider: str
    mitre_category: str
    injection_surface: str
    attack_goal: str
    payload_template: str      # successful payload with {{GOAL}} marker
    for reuse
    confirmed_on: list[str]    # workflow_ids where this was confirmed
    first_confirmed: str      # ISO timestamp
```

This is the mechanism that lets a confirmed `CLAUDE.md` approval manipulation finding on `claude-pr-review` seed a high-priority hypothesis for `claude-issue-triage` automatically — analogous to SIVA's cross-language transfer of vulnerability patterns.

3.6 Report Generator

Input: `list[ValidationResult]`, `EffectivePromptContext` **Output:** markdown + JSON

1. **Workflow Summary** — provider, trigger, tool restrictions, injection surfaces.
2. **Confirmed Vulnerabilities** — MITRE category, surface, payload excerpt, success rate, iteration found, mitigation, residual risk.
3. **Unconfirmed Hypotheses** — ran but did not succeed.
4. **Filtered Hypotheses** — dropped by structural pre-pass or LLM ranker, with reason.
5. **Cost Summary** — total input/output tokens, API cost at current rates, GitHub Actions minutes, wall-clock time.
6. **Reproduction Commands** — `uv run python -m src.benchmark.cli run ...` per confirmed finding.

4. Evaluation Plan

4.1 Main Experiment: Full Workflow Scan

Run the scanner against all 13 workflows with `--hypotheses 12 --max-live 5 --runs-per 3 --iterations 2`. Primary metric: **new confirmed attacks not in the existing 11**. Secondary metrics:

- **Precision** = confirmed / total live-validated hypotheses
- **Recall** = known attacks rediscovered / 11
- **Cost per workflow** = total API spend + GitHub Actions minutes

- **Cost per new confirmed attack** = total cost / new findings

4.2 Baseline Comparisons

Baseline	What it represents
Manual authoring (human)	Current GitInject approach — the scanner's coverage upper bound
<code>zizmor</code>	Static YAML security linter for GitHub Actions; non-LLM, catches misconfigurations
<code>actionlint</code>	Static workflow syntax/semantic checker; non-LLM
Single monolithic hypothesis prompt	Ablation: does per-category specialization improve over one prompt?
Random payload (StaticAttack)	Ablation: does structured hypothesis generation outperform random injection?

The comparison against `zizmor` and `actionlint` is the most important for the paper — it anchors the LLM-based approach against the current practitioner baseline and makes the "LLM finds what static tools miss" claim concrete, mirroring HPTSA's comparison against ZAP/MetaSploit and RepoAudit's comparison against CodeGuru/Meta Infer.

4.3 Ablations

Run on the 4 primary workflows (claude-pr-review, codex-pr-review, gemini-assistant, cline-assistant) to keep cost manageable:

Ablation	Component removed	Expected effect
No LLM ranker (static pre-pass only)	Remove Pass 2 of §3.3	Precision drops; more live runs spent on semantic mismatches
No per-category specialization	Single monolithic hypothesis prompt	Fewer hypotheses per category; MITRE coverage imbalanced
No feedback loop	Remove iterative re-prompting	Confirmed attack rate lower; no recovery from first-pass failures
No cross-workflow memory	Fresh start per workflow	Miss seeded hypotheses; later workflows in same provider take more iterations
Hypothesis count sensitivity	N=6 vs N=12 vs N=20	Diminishing returns in new confirmed attacks; cost scales linearly

4.4 Cost Transparency

Per RepoAudit and HPTSA reporting style, the paper reports for the full scan:

- Total API cost (hypothesis generation + LLM ranker + LLM evaluator calls)
- GitHub Actions minutes consumed across all 13 workflows
- Average wall-clock time per workflow

- Cost per confirmed new attack

5. CLI Integration

```
uv run python -m src.benchmark.cli scan --workflow <workflow_id> [options]
```

Options:

--hypotheses N	Hypotheses per scan, split across 4 MITRE categories (default: 12)
--max-live N	Max hypotheses to validate live, by severity rank (default: 5)
--runs-per N	Runs per hypothesis for confirmation (default: 3)
--iterations N	Hypothesis refinement iterations (default: 2)
--dry-run	Generate and rank hypotheses only, no live runs
--no-ranker	Skip LLM ranker, use structural pre-pass only (ablation)
--no-memory	Disable cross-workflow memory seeding (ablation)
--monolithic	Use single hypothesis prompt instead of per-category (ablation)
--output DIR	Directory to write report
--hypothesis-model M	LLM for hypothesis generation (default: claude-sonnet-4-6)
--all	Run against all workflows in the inventory

Full study scan:

```
uv run python -m src.benchmark.cli scan \
  --all \
  --hypotheses 12 \
  --max-live 5 \
  --runs-per 3 \
  --iterations 2 \
  --output reports/scanner-full/
```

6. New Files

src/benchmark/	
scanner/	
__init__.py	
prompt_extractor.py	# EffectivePromptContext + YAML parsing
hypothesis_generator.py	# per-category sub-prompts + memory seeding
llm_ranker.py	# structural pre-pass + LLM plausibility
scoring	
live_validator.py	# ScenarioCodegen + feedback loop +
ValidationResult	
memory.py	# CrossWorkflowMemory read/write

```
report_generator.py      # markdown + JSON + cost summary
types.py                # shared dataclasses
baselines/
  zizmor_runner.py       # wraps zizmor CLI, normalizes output
  actionlint_runner.py  # wraps actionlint CLI, normalizes output
templates/
  scenario.py.j2         # Jinja2 template for generated scenario files
```

The scanner imports from the existing framework (`BenchmarkRunner`, `AbstractScenario`, `StateEvaluator`, `LLMEvaluator`, `RepoProvisioner`) without modifying them. The only change to existing files is a new `scan` command in `cli.py`.

7. Paper Integration

The scanner adds a new subsection to Section 4 with a named architecture diagram (the pipeline in §2 above), and a fourth contribution bullet:

Vulnerability Scanner. GitInject Scanner takes a workflow YAML as input and autonomously produces a ranked, confirmed attack inventory. It uses per-category hypothesis generation, LLM-based plausibility ranking, live validation with iterative refinement, and cross-workflow memory transfer. Run against all 13 workflows in our study, it confirmed X new attacks not identified in our manual review, at a precision of Y% and a cost of \$Z per workflow.

The appendix on LLM-assisted hypothesis discovery (`app:llm-discovery`) is promoted from a single case study to a systematic evaluation table: per workflow, hypotheses generated / filtered / live-validated / confirmed, broken down by MITRE category, compared against `zizmor` and `actionlint`.

8. Cost Estimation

8.1 Assumptions

Parameter	Value	Notes
Workflows in scope	13	10 Claude, 1 Gemini, 1 Codex, 1 Cline
Hypotheses per workflow	12 (3 per MITRE category)	
Max live runs per iteration	5	By severity rank
Iterations	2	Round 1: top 5; Round 2: replacements for failures
Effective live runs per workflow	~7	Round 1 full (5) + round 2 partial (~2 replacements on average)
Runs per hypothesis for confirmation	3	All must succeed
Hypothesis model	claude-sonnet-4-6	\$3/MTok input, \$15/MTok output

Parameter	Value	Notes
LLM ranker model	claude-sonnet-4-6	Same
GitHub Actions	Public repos	Free minutes

Model pricing used (current rates):

Model	Input	Output
claude-sonnet-4-6	\$3.00/MTok	\$15.00/MTok
claude-haiku-4-5	\$0.80/MTok	\$4.00/MTok
gemini-2.5-flash	\$0.15/MTok	\$0.60/MTok
gpt-4o	\$2.50/MTok	\$10.00/MTok
gpt-4o-mini	\$0.15/MTok	\$0.60/MTok

8.2 Stage 1 — Scanner LLM Costs (Per Workflow)

These are the calls made by the scanner itself, not the workflows under test.

Hypothesis Generator: 4 sub-prompts × 2 iterations, each ~1,500 input + 400 output tokens. Plus ~50% overhead for re-prompting feedback on failures.

- Input: $4 \times 2 \times 1,500 \times 1.5 = 18,000$ tokens
- Output: $4 \times 2 \times 400 \times 1.5 = 4,800$ tokens

LLM Ranker: 12 hypotheses batched into ~3 calls, each ~2,000 input + 300 output tokens.

- Input: $3 \times 2,000 = 6,000$ tokens
- Output: $3 \times 300 = 900$ tokens

Total scanner LLM per workflow: ~24,000 input + 5,700 output tokens **Cost per workflow:** $(24,000 \times \$3 + 5,700 \times \$15) / 1,000,000 = \$0.072 + \$0.086 = \sim\$0.16$ **Cost for 13 workflows: ~\$2.10**

8.3 Stage 2 — Workflow Run Costs (Live Validation)

Each live validation run provisions an ephemeral repo and triggers the real provider workflow, which makes its own LLM API calls. These are the costs paid to the provider whose workflow is under test — distinct from the scanner's own LLM spend in §8.2.

These are agentic runs, not single-turn calls. The model receives the full growing context on each tool-call turn. A typical PR review run involves 3–6 turns (read diff → read files → post comment). Token estimates accordingly use a **per-run total across all turns**, not a single call:

Scenario	Input tokens (total across turns)	Output tokens
Conservative (short diff, 3 turns)	15,000	2,000
Realistic (typical diff, 5 turns)	35,000	4,000

The table below uses the realistic estimate. Conservative estimates are shown in parentheses.

Provider	Model	Cost/run	Runs (7 hyp × 3 confirm)	Workflows	Total
Claude	claude-sonnet-4-6	\$0.165 (\$0.075)	21	10	\$34.65 (\$15.75)
Gemini	gemini-2.5-flash	\$0.008 (\$0.003)	21	1	\$0.17 (\$0.06)
Codex	gpt-4o	\$0.128 (\$0.058)	21	1	\$2.69 (\$1.22)
Cline	gpt-4o-mini	\$0.008 (\$0.003)	21	1	\$0.17 (\$0.06)
Total					\$37.68 (\$17.09)

Cost/run derivations (realistic):

- Claude: $(35,000 \times \$3 + 4,000 \times \$15) / 1,000,000 = \$0.105 + \$0.060 = \$0.165$
- Gemini: $(35,000 \times \$0.15 + 4,000 \times \$0.60) / 1,000,000 = \$0.005 + \$0.002 = \$0.008$
- Codex: $(35,000 \times \$2.50 + 4,000 \times \$10) / 1,000,000 = \$0.088 + \$0.040 = \$0.128$
- Cline: $(35,000 \times \$0.15 + 4,000 \times \$0.60) / 1,000,000 = \$0.005 + \$0.002 = \$0.008$

Claude dominates because there are 10 Claude workflows and sonnet-4-6 is the most expensive model in scope. See §8.7 for levers to reduce this.

8.4 Stage 3 — LLM Evaluator Costs

Used only for judgment-manipulation hypotheses where success is semantic ([LLMEvaluator](#)). Estimated 2–3 confirmed judgment-manipulation findings across all workflows. Judge model: Gemini 3.1 Pro Preview (~\$1.25/MTok input, \$5.00/MTok output).

- Per eval call: $\sim 3,000$ input + 300 output = $\$0.004 + \$0.0015 = \$0.006$
- 3 findings × 3 runs × $\$0.006 = \sim \mathbf{\$0.05}$ (negligible)

8.5 Ablation Costs

Ablations run on 4 primary workflows (all Claude) × 4 ablation conditions. Each ablation is a full scan at the same parameters.

- Scanner LLM per ablation condition: 4 workflows × $\$0.16 = \0.64
- Workflow runs per ablation condition: 4 workflows × 21 runs × $\$0.165 = \13.86
- 4 ablation conditions: $4 \times (\$0.64 + \$13.86) = \sim \mathbf{\$58.00}$

8.6 Summary

Component	Cost (realistic)	Cost (conservative)
Scanner LLM (hypothesis gen + ranker), 13 workflows	\$2.10	\$2.10
Workflow run costs, all providers	\$37.68	\$17.09
LLM evaluator	\$0.05	\$0.05
Ablations (4 conditions × 4 Claude workflows)	\$58.00	\$26.36
Total	~\$97.83	~\$45.60

GitHub Actions minutes: 13 workflows × 7 hypotheses × 3 confirmation runs × ~4 min/run = ~1,092 minutes ≈ 18 hours. Public repositories get free runner minutes, so this has no dollar cost but sets the wall-clock expectation. With runs parallelized across workflows, elapsed time is ~3–4 hours.

Cost per new confirmed attack (target metric): If the scanner confirms 5 new attacks beyond the existing 11, the main experiment cost (excluding ablations) per new finding is $\$39.83 / 5 \approx \text{\$8 per new finding}$ (realistic) or $\$19.24 / 5 \approx \text{\$3.85}$ (conservative). Both are in the same order of magnitude as RepoAudit's \$2.54 per project and directly reportable in the paper.

8.7 Sensitivity

The dominant cost driver is Claude workflow runs ($\$34.65 / \$37.68 = 92\%$ of main experiment workflow costs), because there are 10 Claude workflows and claude-sonnet-4-6 is the most expensive model in scope. Three levers to reduce this if budget is tight:

1. **Collapse similar Claude workflows first.** claude-pr-review, claude-pr-review-paths, and claude-pr-review-authors are structurally near-identical; the scanner's cross-workflow memory means the second and third scan in this group produce far fewer new hypotheses and require fewer live runs. Running them sequentially with memory enabled could reduce effective live runs by ~30% for the Claude group.
2. **Use claude-haiku-4-5 as the victim model for preliminary iterations.** Cost per run drops from \$0.165 to ~\$0.044 ($35,000 \times \$0.80 + 4,000 \times \$4.00 / 1,000,000$). Use sonnet only for final confirmation runs of hypotheses that survive the haiku pilot. This cuts Claude workflow run cost from \$34.65 to ~\$12–15.
3. **Reduce confirmation runs from 3 to 1 for the ablation sweep.** Since the ablations measure relative precision/recall across conditions (not absolute confirmation), single runs are sufficient for comparison. This cuts ablation cost by ~66%.

9. Implementation Order

1. `types.py` — all shared dataclasses including cost fields and `MemoryEntry`. (Budget checkpoint: verify model pricing against current Anthropic/OpenAI/Google rate cards before the full scan.)
2. `prompt_extractor.py` — YAML parsing and `EffectivePromptContext`. Testable offline.
3. `hypothesis_generator.py` — four category sub-prompts + memory seeding. Testable offline.
4. `llm_ranker.py` — structural pre-pass rules + LLM plausibility scorer. Testable offline.
5. `memory.py` — read/write `scanner-memory.json`. Pure I/O, unit-testable.
6. `baselines/zizmor_runner.py` and `actionlint_runner.py` — wrap CLI tools, normalize output.

7. `templates/scenario.py.j2` + `ScenarioCodegen` in `live_validator.py` — validate against one known attack first before running novel hypotheses.
8. Feedback loop in `live_validator.py` — wire `update(score)` pattern into the iteration loop.
9. `report_generator.py` — markdown + JSON + cost summary.
10. `cli.py` — wire everything behind `scan`, with all ablation flags.
11. **Pilot run** on `gemini-assistant` and `codex-pr-review` — validate the full pipeline end-to-end and check for new findings before committing to the full scan.
12. **Full scan** against all 13 workflows with memory enabled; run ablations on the 4 primary workflows.
13. Add scanner results, baseline comparison table, and ablation table to `paper/neurips2026/`.